# 4.通识

## 一、主要问题

请根据以下问题把握文章的重点：

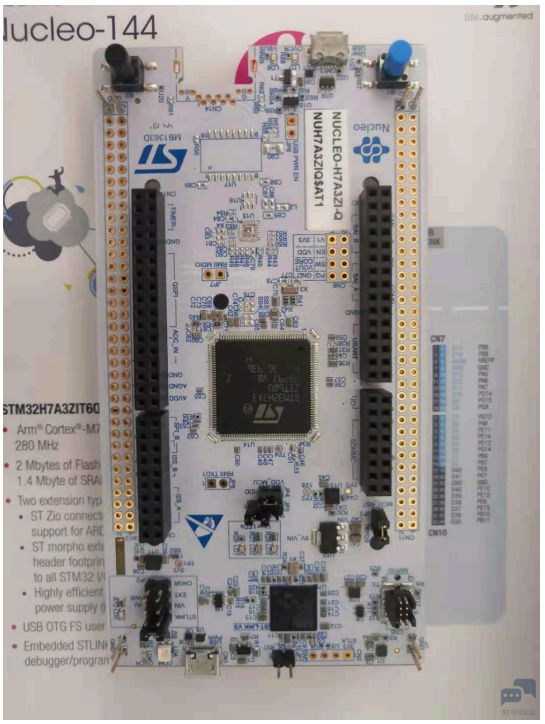
- PC中的数据存储在内存空间中吗？手机中的数据存储在内存空间中吗？MCU中的数据存储在内存空间中吗？
- 一个能存储4Byte的存储设备中，能够记录的10进制的数字的范围是多少
- **你能找到16进制与2进制之间转换的规律吗？**
- **你知道什么是掩码和位运算吗？如何利用掩码和位运算只读取某个Byte中的低四位**
- 0x1000和1000有什么区别？它们两个相等吗？
- 你认为代码控制硬件时中间的桥梁是什么。
- 完成CubeMX基础配置

## 二、单片机简介

生活里很多设备内部都藏着一块小小的芯片：微波炉控制加热、洗衣机执行洗衣流程、电子钟计时、智能小车接收传感器信号，这些设备不会搭载Windows、安卓系统，但它们也需要一套“大脑”执行控制逻辑。这块专门用来做控制的芯片，就是**单片机，也叫MCU（微控制器）**。

普通电脑，CPU、内存条、硬盘全部是互相独立的零件，需要插在主板上接线才能工作。但单片机不一样：CPU、临时内存（RAM）、掉电保存程序的Flash、还有控制外设的硬件电路全部封装在同一块芯片内部。**一颗芯片，通电就可以独立完成控制任务，不需要外接内存条、硬盘。**

通俗来讲，单片机就是**把一整台微型计算机压缩、集成到一颗小小的芯片里面**。除此之外，MCU的控制往往需要各种硬件的支持，如用于计时的定时器电路，用于模拟信号产生的DAC电路和将模拟信号转换为数字信号的ADC电路，用于真随机数产生的RNG电路等等，这些电路是CPU的外围设备，统称为外设。这些外设的存在，让单片机的功能不只是进行数据运算，能够对外界产生信号或者接收外界信号，集成度更高。CPU通过这些外设里的寄存器控制外设运行和读取外设中的数据。




NUCLEO-144开发板，最大的芯片是MCU STM32H7A3

## 单片机组成与运行原理简单叙述

由于单片机实际上是一个完整的计算机，因此单片机的组成和运行原理与PC比较相似。

单片机上电后，CPU 从 Flash 中按地址依次读取、解析并执行指令，默认遵循**顺序执行**的逻辑，逐行运行主程序里的代码。当遇到 if、循环、函数调用时会发生跳转，不再按原有顺序执行；若触发中断，CPU 会立刻暂停当前任务，优先执行中断服务程序，处理完毕后再回到断点继续原来的流程。程序运行时临时数据存放在 RAM 中，GPIO、定时器、串口等片上外设可配合 CPU 完成各类硬件控制，部分外设还能依靠 DMA 独立传输数据，减轻 CPU 负担。

完整过程是这样：我们在电脑上写C语言源代码；编译器把人类看得懂的C代码翻译成单片机CPU看得懂的0/1二进制指令；再通过下载工具，把这一堆二进制指令写入单片机的Flash存储里面；之后只要给单片机上电，CPU就会自动从存储中逐条读取指令，不停执行。

## CPU内部结构与工作流程

CPU内部可以简单分为三部分：控制单元、算术逻辑单元ALU、内部寄存器。

CPU执行程序，会无限循环一套固定五步流程，每完整走完一轮，就需要依靠时钟信号触发。

1. **取指：** 从Flash或者内存当中，读出接下来要执行的那一条指令；
2. **译码：** 控制单元解析这条指令，搞清楚这条指令想要做什么：是做加减法？还是读写某个地址的数据？
3. **执行：** 交给算术逻辑单元完成运算或者逻辑判断；
4. **访存：** 如果这条指令需要读写内存，就通过总线完成内存访问；不需要读写内存就跳过这一步；
5. **写回：** 把运算得到的结果，存回到CPU内部寄存器，或者写入内存。

# 存储器与寻址

## 存储器分两大类，核心差异就是断电之后数据会不会消失。

- **易失存储器 SRAM（也常叫RAM）：断电，数据全部消失**

它就相当于电脑的内存条。单片机程序运行的时候，所有我们定义的变量、函数里面的临时计算结果，全部放在RAM中。一旦断电，RAM里面所有内容直接清空，不会留下任何数据。

- **非易失存储器 Flash：断电，数据不会丢失**

类似电脑硬盘，专门存放编译完成的程序代码。就算关掉电源，写进去的程序还保留在芯片里。

下载代码后，程序直接存放在片内Flash。上电之后，**不需要把程序复制到RAM**，CPU直接从Flash读取指令执行；RAM只负责存放运行过程中产生的变量、临时数据。

## 地址、总线、寻址空间

芯片里面可能同时存在很多块存储，还有一堆硬件外设。CPU想要读写某一处的数据，它怎么知道该去找哪一块?

答案就是依靠**地址**。每一个可以访问的位置，都会分配一个独一无二的数字编号，这个编号就是地址。

硬件上依靠两类总线完成数据交互：

**地址总线：** CPU把要访问的地址编号通过这一组电线发送出去，告诉硬件“我要访问哪一个位置”。地址总线有多少根导线，直接决定CPU最多能访问多大的存储空间。

**CPU是多少位，不等于地址总线的根数**。经典例子：8086是16位CPU，但是它拥有20根地址线。

**数据总线：** 真正用来搬运数据。读的时候硬件把数据送回CPU，写的时候CPU把数据发送给硬件。总线宽度决定一次可以传输多少位二进制。

> 单位换算（大小写千万不能搞混，b和B差整整8倍）
>
> - `b` = bit 比特：1个二进制0或者1
> - `B` = Byte 字节：1 Byte = 8 bit
> - 1KB = 1024 Byte；1MB = 1024 KB；1GB = 1024 MB

嵌入式开发描述地址几乎全部使用十六进制，以 `0x` 作为开头，例如 `0x20000000`，十六进制书写长地址更加简短直观。

## 寻址空间不全是真正的内存

CPU所有可以通过地址访问到的全部范围，叫做**寻址空间**。但地址对应的并不都是实实在在的内存芯片。

在整片地址空间里面，一部分地址指向真正的RAM内存，一部分指向Flash程序存储；还有很大一部分地址，**并没有对应的存储芯片，而是映射到各个外设的寄存器上面。**

以STM32芯片为例，`0x40000000 ~ 0x5FFFFFFF` 这一大段地址，全部是外设寄存器的映射地址。

> ✨关键：我们操控GPIO、串口、定时器，底层本质就是CPU读写这一段地址上的寄存器。**内存映射寄存器，就是软件控制硬件的中间桥梁。**

> 补充提醒：手册给出的地址区间只是逻辑划分，不代表芯片物理上真的拥有这么大内存。比如手册写RAM的地址范围是0x20000000开始，意思是所有RAM物理单元都必须落在这个区间；实际芯片真实RAM大小往往只有几十KB。

## 片内总线分级

CPU运行速度非常快，但是外设（GPIO、串口）的响应速度要慢得多。如果全部设备共用同一套总线，高速的CPU会被低速外设拖慢整体速度。

所以芯片内部总线会做分级，高速设备和低速设备分开，中间用“桥接模块”做信号转发。

对于Cortex-M内核单片机：

- **AXI、AHB总线**：高速，用来连接CPU、RAM这类高速部件
- **APB总线**：低速，绝大多数外设（GPIO，串口，定时器）都挂载在APB总线上。0x4000 0000 - 0x5FFF FFFF这段外设内存空间一般会被分配到APB总线上。

### 寄存器

寄存器是单片机内部用于 CPU 和硬件外设交互的高速存储单元，是操控硬件的底层接口。单片机上的 GPIO、定时器、串口等每一个外设，都配套一组专属寄存器，CPU 通过对寄存器执行读写操作，完成硬件配置与状态采集。我们日常使用的 HAL 库函数，本质上也是对寄存器的读写进行封装。

寄存器通常以 32 位为基本单位，每一个比特位都对应一项独立功能。向寄存器的对应位写入 0 或 1，就可以开启、关闭外设或是修改工作参数；读取寄存器的数值，则能够获取外设当前的实时状态。它和内存 RAM 有着明确区别：RAM 用来存储程序运行中的变量数据，而寄存器直接绑定硬件电路，读写时会立刻引起硬件行为变化，访问速度也远高于内存。

以 GPIO 控制 LED 为例：GPIO 存在专门的输出数据寄存器，将寄存器对应引脚的比特位置 1，引脚就输出高电平，LED 点亮；置 0 则输出低电平，LED 熄灭。理解寄存器，是脱离库函数封装、看懂底层驱动、调试时序与硬件异常的基础。

## 下载：内存布局与下载器

解决完让CPU运行的问题之后还需要解决的问题是如何让代码正确地放置在Flash区域而不是其他位置。Flash的位置并不是由ARM架构决定的，而是由芯片研发厂商决定，STM32系列MCU由意法半导

体（ST）设计制造，ST将其Flash的位置规定在0x0800 0000开始，通过一个叫做链接文件的文件可以控制代码分布位置。

为了将程序下载到芯片中，比较好的方式是使用下载器，ST为STM32系列开发了STLink下载器，将单片机与PC连接在一起，能够将程序从PC中下载到单片机。除了STLink，还有SEGGER公司的JLink，ARM官方开源的DAPLink等。

# 三、关于STM32

- **STM32 是 意法半导体（STMicroelectronics） 推出的 32 位 ARM Cortex-M 内核微控制器（MCU）系列**。
- 采用 ARM 授权的 Cortex-M 系列 CPU 内核，集成了 **CPU + Flash + SRAM + 各类外设**，可独立运行。
- 定位是通用型 MCU，覆盖从低功耗到高性能，从入门级到工业级的应用。

## 内部资源

**CPU 内核**

- 早期：Cortex-M0/M3/M4
- 后来：Cortex-M7（高性能）、Cortex-M33（安全与低功耗）
- 最新：Cortex-M55、双核 M7+M4 组合

**存储**

- 内部 Flash（几十 KB 到几 MB）
- 内部 SRAM（几 KB 到上 MB）

**外设**

- 通用 I/O（GPIO）
- 通信接口：USART、SPI、$I^2C$、CAN、USB、Ethernet、SDIO、QSPI
- 模拟外设：ADC、DAC、运算放大器、比较器
- 定时器：基础定时器、高级定时器（支持 PWM、死区控制、编码器接口）

# 四、开发STM32

之前所有的内容看不太懂没关系，那只是为了更好地理解开发的时候都在做什么。不了解也可以开发单片机。

重新叙述一下单片机开发的大概思路：让CPU执行指令，访问内存和控制外设，实现与外部的输入与输出。因此我们需要软件编写代码，转换成CPU指令，也需要用软件配合硬件将这些指令复制到单片机的Flash中存储起来。

开发单片机就需要得到能够让单片机运行的程序。最开始，一个程序并不是很复杂，人们使用机器码开发，后来发现机器码编写更复杂的功能十分麻烦，于是使用汇编帮助人们编写程序，使用汇编器转为机器码。随着控制程序复杂程度的提高，汇编也不能够满足需求，于是人们使用高级编程语言，用编译器将高级语言代码文件编译为程序。现在，人们通常将大部分工程中都会用到的部分打包成库，并且使用工具自动生成大部分工程通用的模板，简化工程的创建和开发，提高开发效率。

工程的开发则是需要一套完整的流程，包括代码编辑，代码编译，版本控制，调试与下载。这些需要不同的软件工具完成，如代码编辑可以用任何的文本编辑器，编译选择对应的编译器（对于STM32有gcc, AC5/6, LLVM等），主流的版本控制git，以及openocd进行调试和下载。人们在开发的过程中感觉到这些功能需要特定软件提供十分繁琐，因此将这些开发所需要的环境（如编译工具，调试工具，构建工具等）打包成一个集成软件，这个软件称为集成开发环境（IDE），主流的IDE有Keil, IAR, STM32CubeIDE，VS Code, JetBrains CLion等软件，这些软件能够完成代码编辑，项目构建，代码编译，下载与调试，集成git之后进行版本控制。

## CubeMX配置



注：之后所有的安装路径、文件名称、工程名称不能使用中文或中文字符，只能使用英文字符，数字和英文字符（注意所有的英文字符是半角符号，如果不清楚什么是英文字符，关掉输入法。）

注：所有的过程中如果出现问题，建议上Bing或者其他搜索引擎搜索报错信息

注：这东西大部分是英文，没有汉化

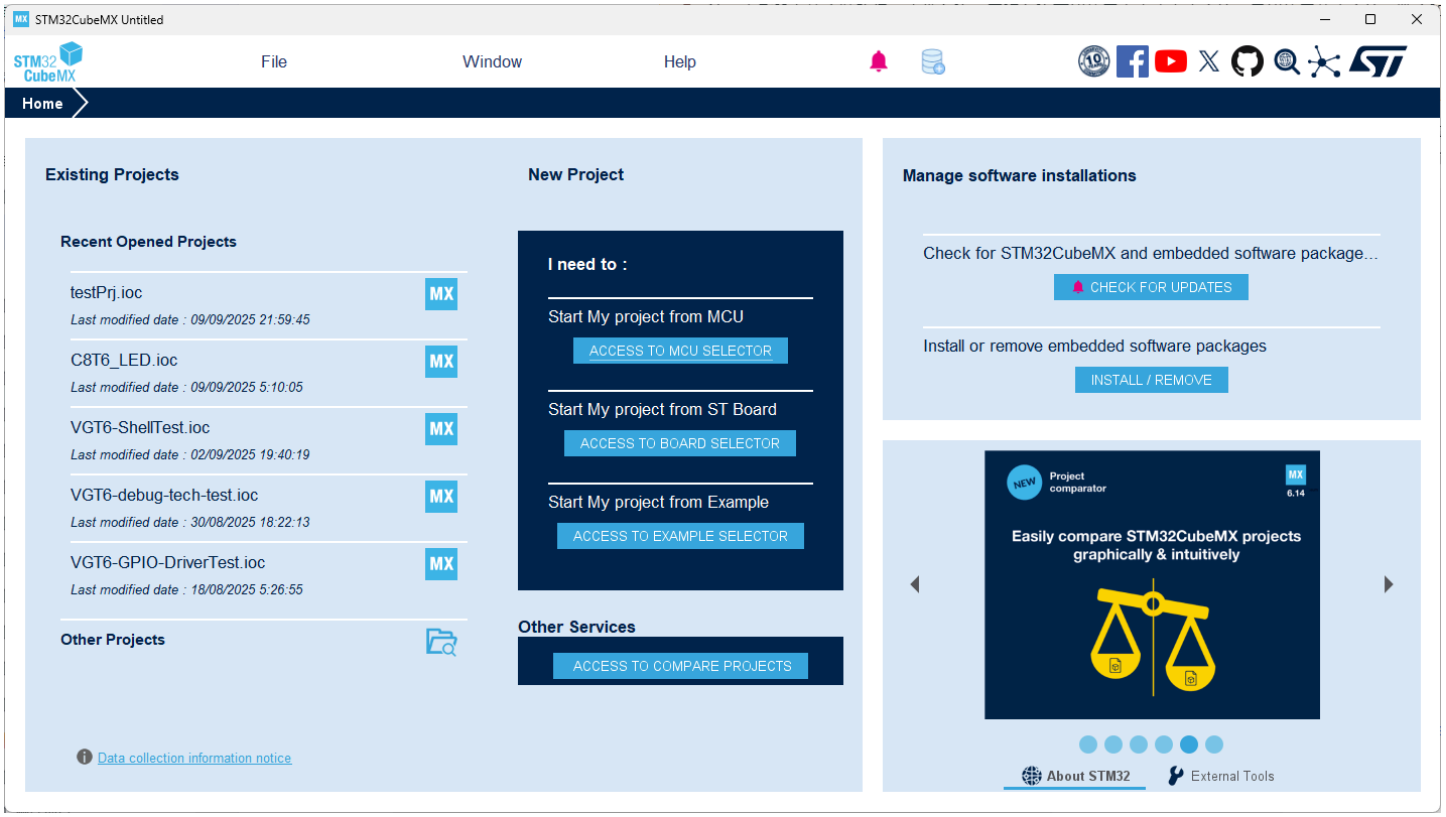
对于初学者，我们选择使用**STM32CubeMX**建立工程，使用**CLion**进行开发

为了简化STM32芯片的开发，ST推出了STM32CubeMX通过图形化界面的方式配置单片机，并且可以自动生成初始化和配置代码。并且将HAL库纳入工程中。使用的时候直接在初始化程序完成之后使用HAL库完成相应功能即可。这里用STM32F103ZET6举例。

ST官网获取https://www.st.com/en/development-tools/stm32cubemx.html，因为是外国网站所以下载速度可能比较慢，下面是已经下载好的安装包

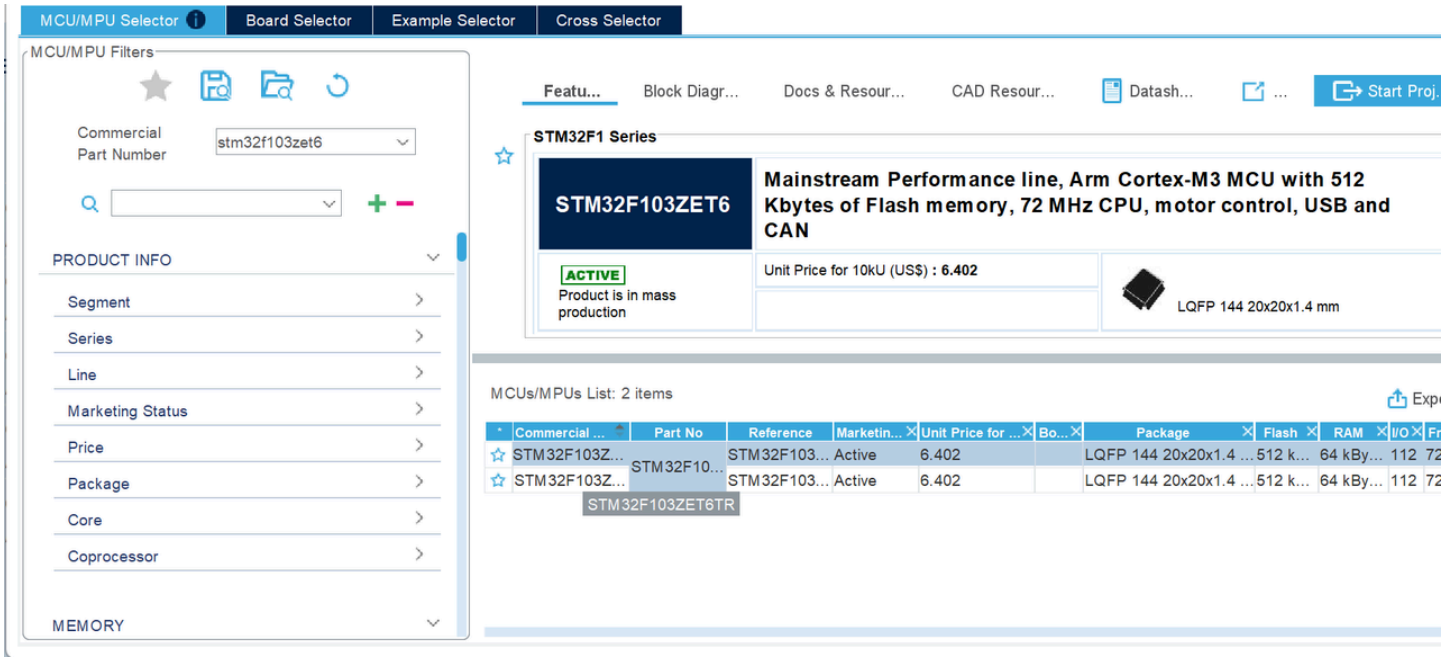
> 通过网盘分享的文件：SetupSTM32CubeMX-6.18.1.exe
>
> 链接: https://pan.baidu.com/s/15xyqeD1dv8FBZhnGdoExoQ?pwd=hrba 提取码: hrba
>
> --来自百度网盘超级会员v3的分享

启动点击 ACCESS TO  MCU SELECTOR 新建工程

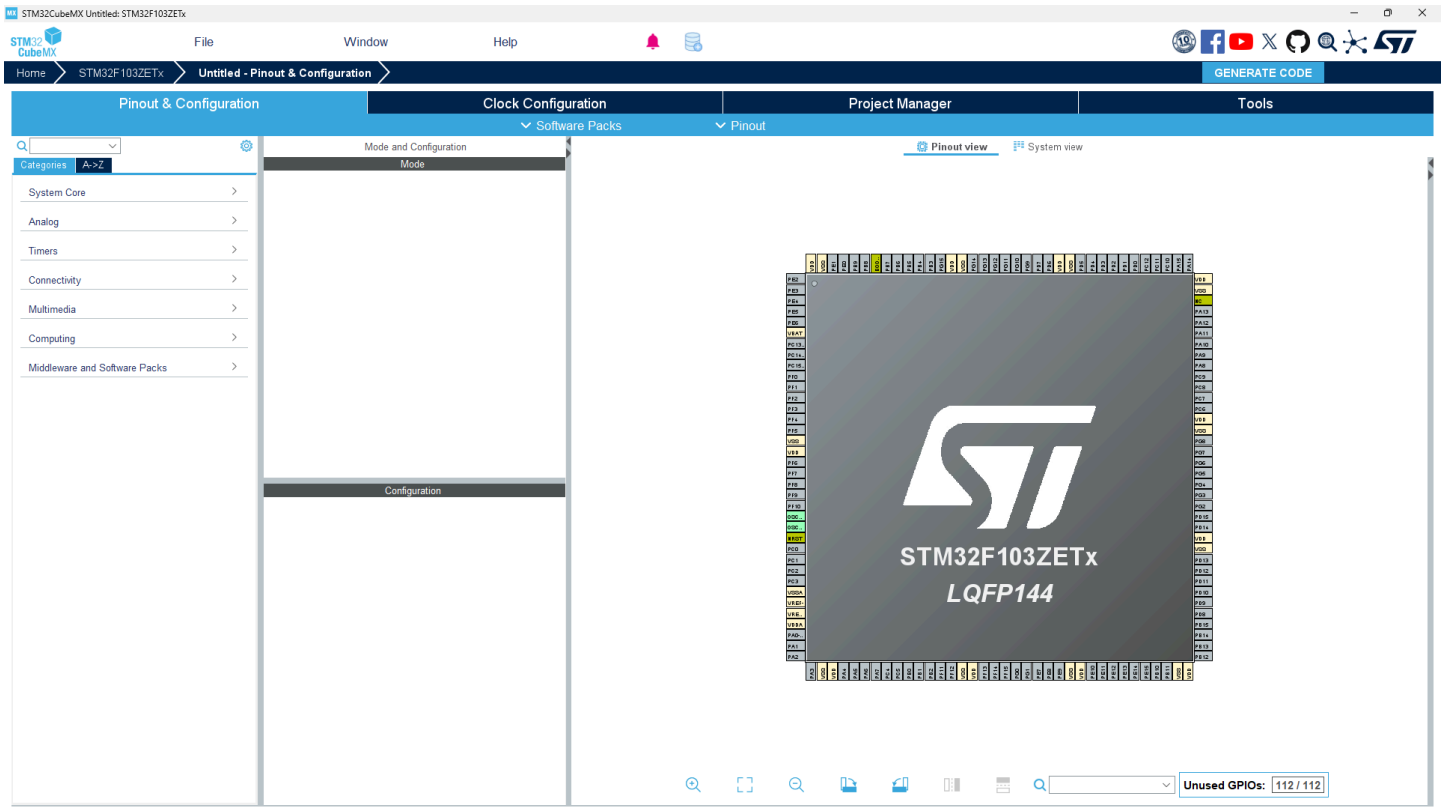



Home界面中最左侧是最近打开的工程，中间是工程创建，右侧是库软件包管理。CubeMX的工程文件就是.ioc文件。

New Project中选择新建工程，三个选项分别是选择MCU，选择开发板和选择例程，咱们一般是直接从MCU开始，使用自己的开发板，因此选择ACCESS TO MCU SELECTOR。




搜索 STM32F103ZET6



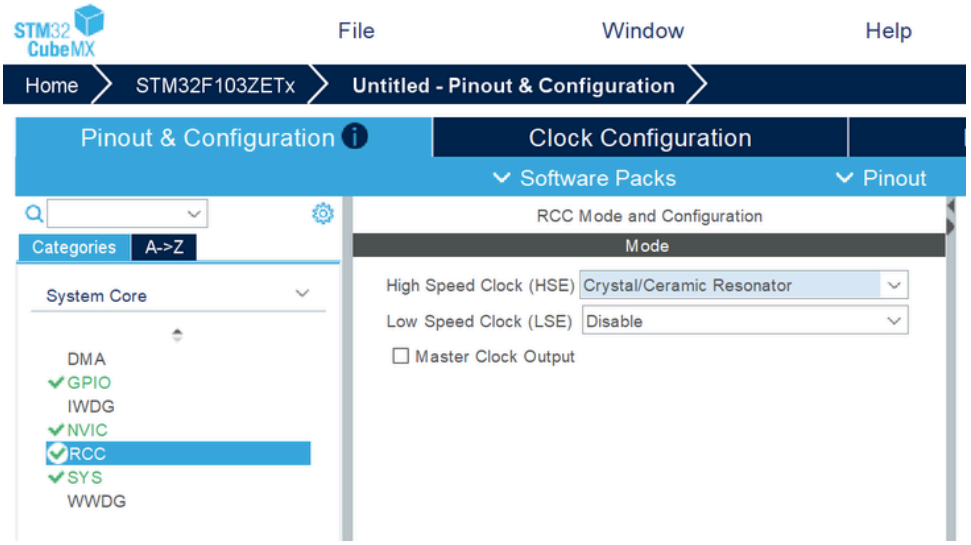

可以看到配置界面中有四个界面从左到右分别是：

- 引脚布局和配置
- 时钟配置
- 项目管理
- 工具

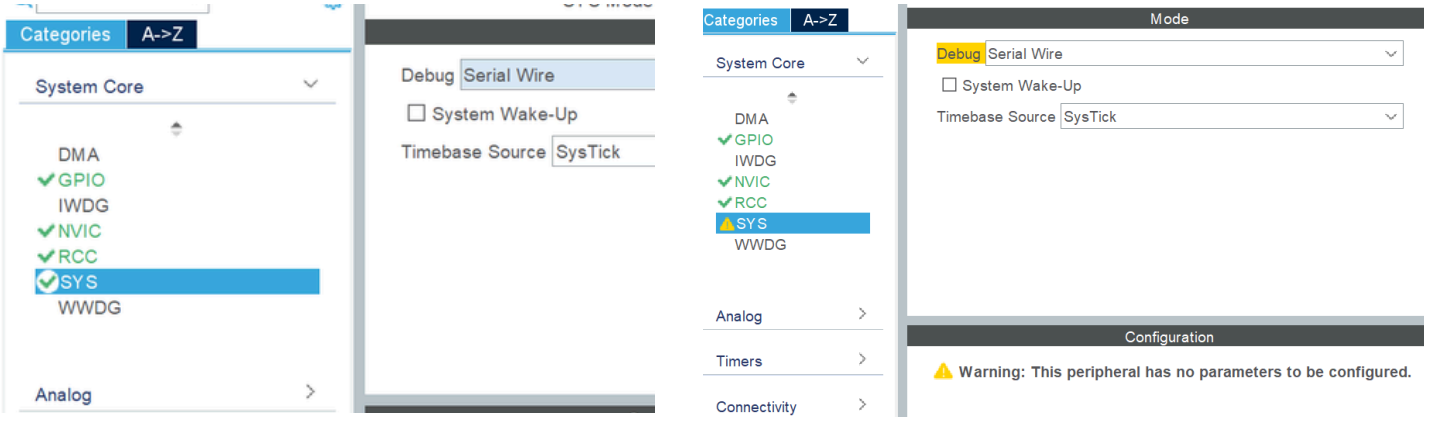
## 基础配置

从新建工程后的配置顺序来讲，所有工程都强烈建议一开始就配置SYS和RCC。SYS配置的是debug接口和系统的计时用外设，RCC配置的是单片机的时钟源。

将 High Speed Clock(高速外部时钟)改为Crystal/Ceramic Resonator(晶振)



启用 Serial Wire（简称 SWD，Serial Wire Debug），如果出现如下警告忽略即可，不会用会导致Debug的代码下载进去后，芯片会关闭下载接口，




选择之后芯片管脚发生了什么：

- **PA13 → SWDIO（双向调试数据）**
- **PA14 → SWCLK（调试时钟）**

> ✅ 这两个引脚**不再能当做普通 GPIO、串口等其他功能用**，CubeMX 会自动把这两个脚锁定为调试复用功能✅ 剩下原本 JTAG 要用的 PA15、PB3、PB4 **全部释放，可以自由当普通 IO 使用**（和直接选 JTAG 最大区别）

这里说一下CubeMX中可以开启的Debug选项：

- **JTAG接口**

  JTAG 是一种标准化的 IEEE 1149.1 测试接口

标准 JTAG 需要 4~5 根信号线：

| 信号 | 功能 |
| --- | --- |
| TCK | Test Clock，调试时钟信号 |
| TMS | Test Mode Select，模式选择 |
| TDI | Test Data In，数据输入 |
| TDO | Test Data Out，数据输出 |
| nTRST (可选) | Test Reset，复位 JTAG 状态机 |

JTAG with Trace Synchro (1 bit / 2 bits / 4 bits)

- 在 JTAG 基础上，增加 Trace 数据输出接口（即 ETM Trace）。
- 1/2/4 bits 表示 Trace 数据线数量
- 主要用于高性能调试、跟踪 CPU 指令执行路径（常用在 H7/F7 系列）

- **SWD（Serial Wire Debug）**

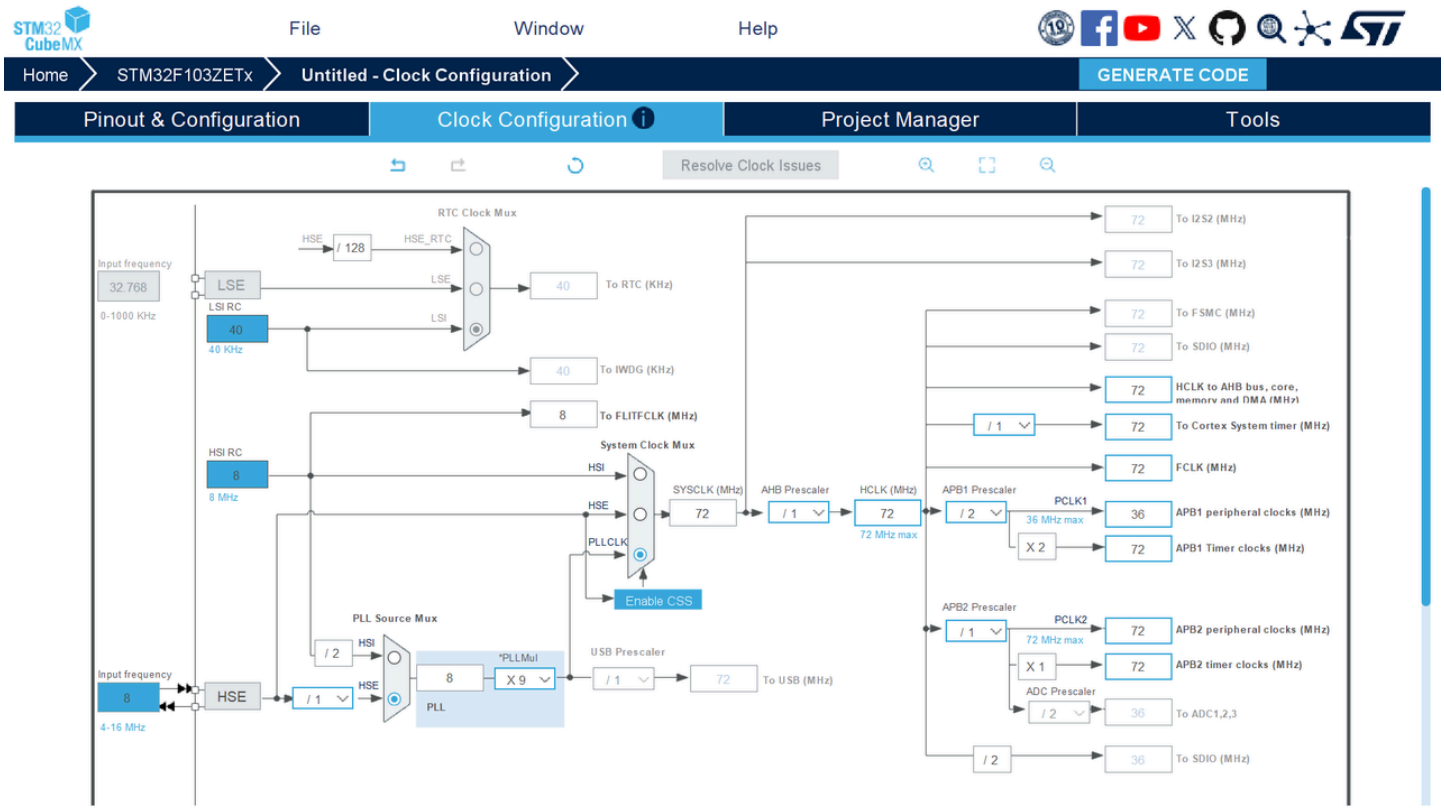
  是 ARM 公司推出的一种调试接口，用于 Cortex-M 系列 MCU。它是 JTAG 的简化版本，专门针对嵌入式低引脚设备设计的。

| 引脚 | 作用 |
| --- | --- |
| SWDIO(简写版有DIO, D, SDO等) | 双向数据线 |
| SWCLK(简写有SCL, SCK, C等) | 时钟线 |

Trace Asynchronous Sw/Trace Synchro Sw (1 bit / 2 bits / 4 bits)

- Asynchronous表示异步传输，多一根SWO线用于Trace功能。
- Synchro Sw为同步传输，与JTAG配置下的Trace Synchro相同

接下来切换到Clock Configuration界面，这里展示的是芯片的时钟树，HCLK(芯片的主频)最大可以设置为72MHz，主频和单片机的性能直接相关，一般来说主频越高越好，我们用的芯片最高频为72Mhz，所以直接拉满；而时钟源则是越稳当越好，板载的时钟源有芯片自带的rc震荡电路与外部的晶振电路（HSE），故选取HSE；在不涉及tim，usb，iic，spi这些对总线频率有特殊要求的外设时，总线频率选择cubemx自动求解出来的就行。



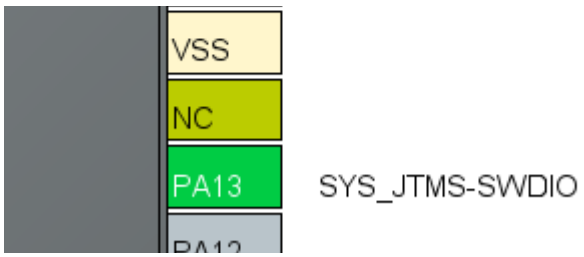

再观察一下时钟树，最左侧的竖线区分芯片内和芯片外，HSE就是外部高速时钟。梯形的选择器选择了HSE，意思是PLL倍频器的输入是HSE的8MHz输入，倍频器将HSE的8MHz翻9倍，选择器选择了PLLCLK，就可以得到APB的最大时钟频率72MHz。但是APB1总线上的外设不需要这么高的频率，

因此APB1 Prescaler即分频器会再次将HCLK二分频，APB1总线上所有外设的输入频率都是36MHz，而APB1总线的定时器的输入频率固定会被倍频，因此还是72MHz输入。APB2总线上最大频率也是72MHz，因此可以不分频。

## 针脚布局

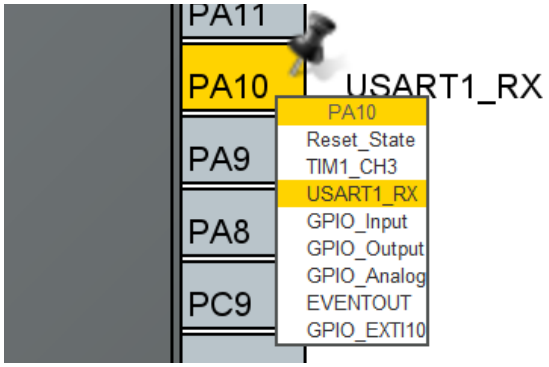
引脚布局和配置中可以用来配置需要启用的外设，以及在这里配置参数，生成工程之后会自动生成这些外设的初始化代码。

右侧的Pinout view中可以看到这款芯片的引脚定义，被使用的引脚会从灰色变为绿色。启用外设的时候就会使用。

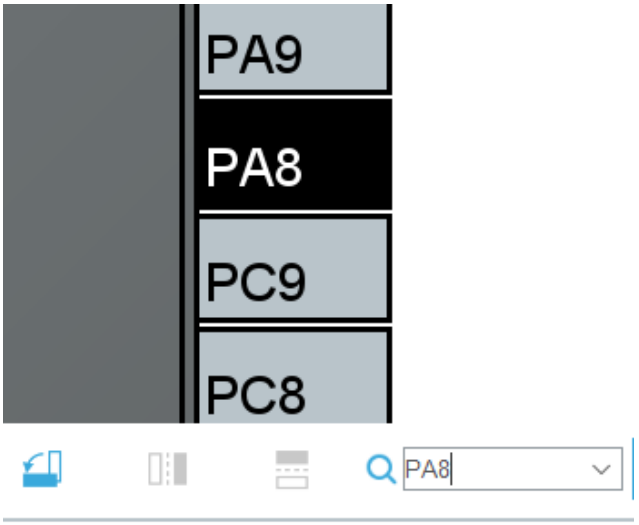



开启了Serial Wire调试接口之后会使用这两个引脚

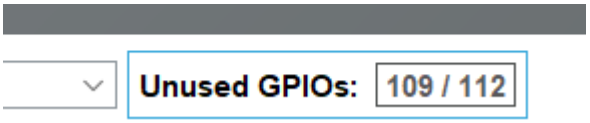
对于没有被使用的引脚，可以点击引脚，然后从下拉栏中选择功能，这样的引脚会被锁定，除非手动清除(选择Reset_State)。如果这个功能需要启用其他外设，引脚会变成黄色。



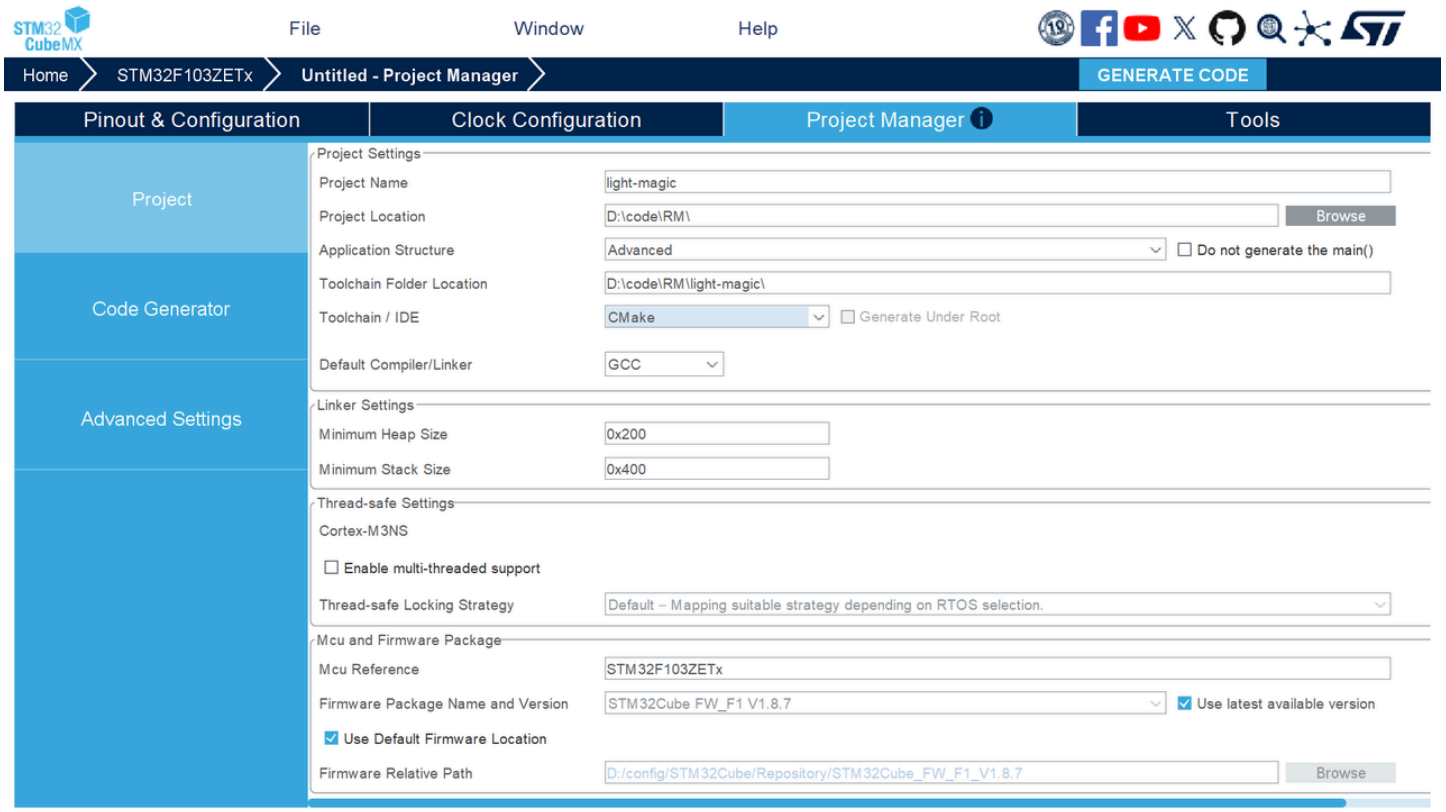

右下角的搜索框可以搜索引脚（不要带空格以及不要使用中文输入法打字母），被搜索中的引脚会黑白闪烁，



右下角会显示未使用的引脚数量




## 项目管理




项目管理界面左侧也有三个选项卡：项目、代码生成器以及高级设置

- 项目（Project）

  这一栏需要注意的就是Project Settings、Linker Settings这两个设置内容。

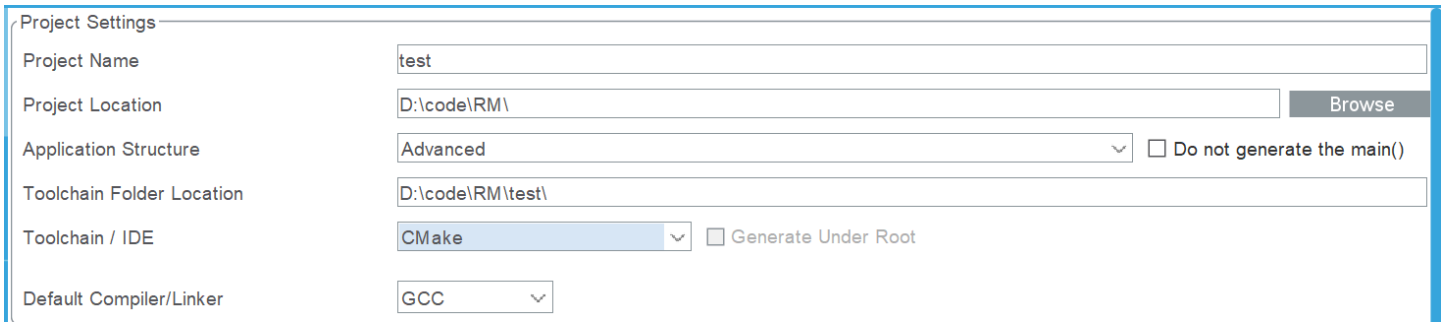
  Thread-safe Settings是多线程安全，在使用FreeRTOS之前不会用到（使用之后一般也不会用到）

  Mcu and Firmware Package是MCU的固件包，一般不动

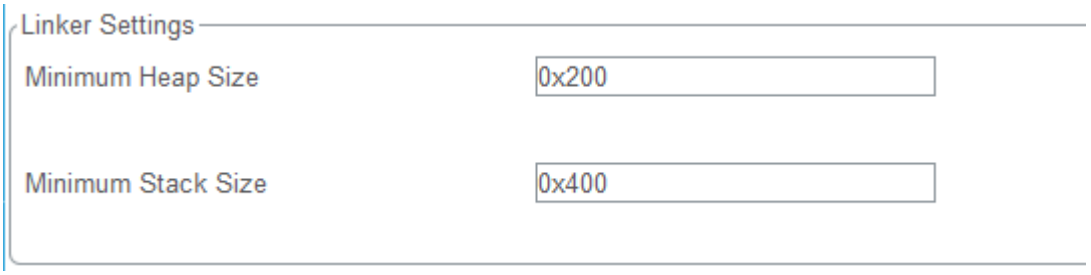
  **Project Settings**需要设置项目名称（再次强调不要用中文和中文标点符号，如果不确定，输入的时候关闭输入法，使用ASCII码字符）

  **Project Location即**项目的位置，设置在你能够找到的位置（不要用中文）

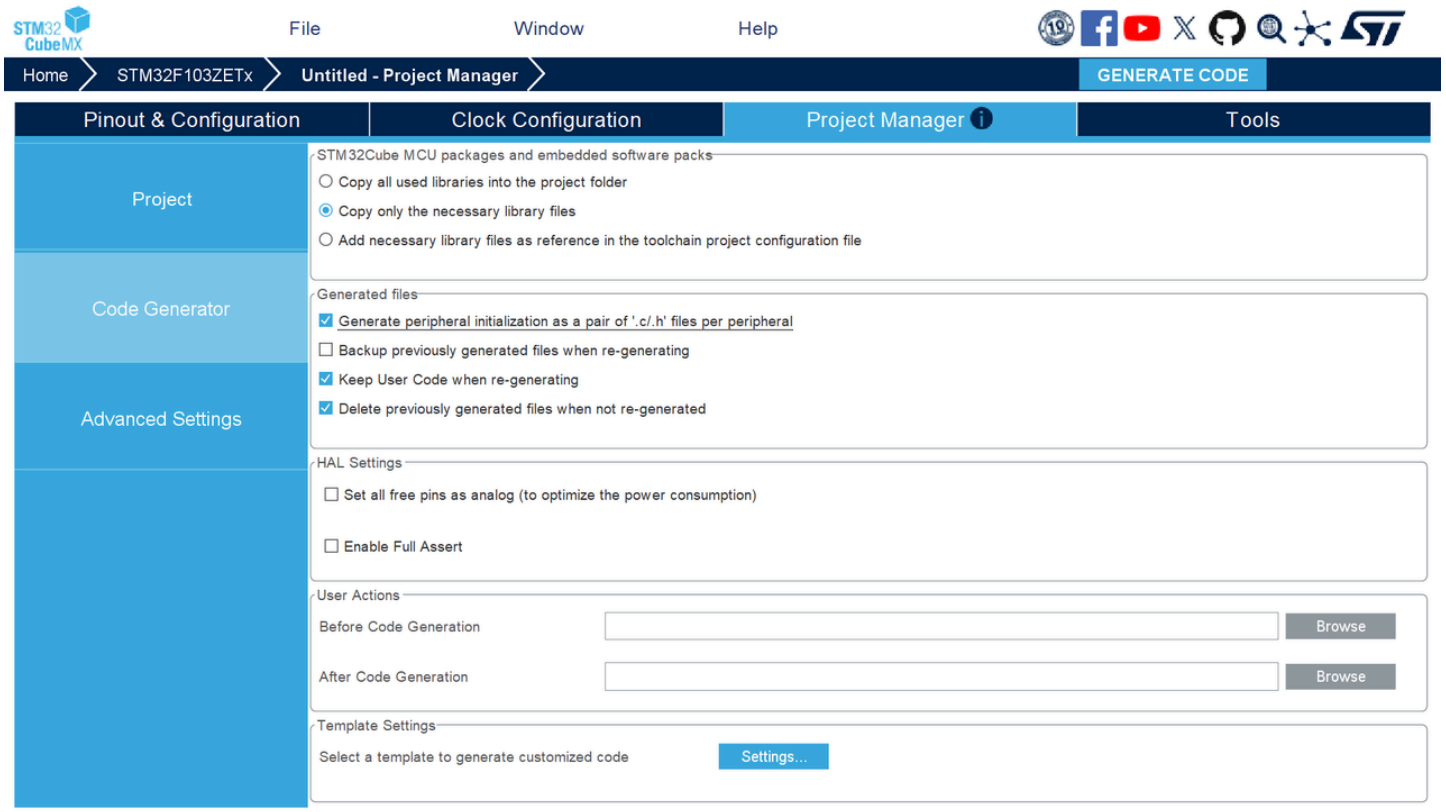
  Toolchain/IDE设置工程的工具链/IDE，使用**CMake**




Linker Settings这一栏配置的是最小堆栈空间，学习阶段一般不需要扩充




- 代码生成器（Code Generator）




第一个栏中配置的是如何处理库文件，三个选项分别为：

- **Copy all used libraries into the project folder**
  将HAL库的所有.c和.h都复制到所建工程中
  优点：工程独立，换电脑、拷贝给别人直接编译，不依赖本地 Cube 库，这样如果后续需要新增其他外设又可能不再用STM32CubeMX的时候便会很方便
  缺点： 体积大，编译时间相比于其他选项非常长

- **Copy only the necessary library files**
  只复制所需要的.c和.h
  优点：体积相对小，编译时间短，并且工程可复制拷贝
  缺点： 新增外设时需要重新用STM32CubeMX导入
- **Add necessary library files as reference in the toolchain project configuration file**
  不复制任何库文件到工程里，只是在工程配置里直接引用 Cube 安装目录下的原始库文件（软链接 / 引用）
  优点：体积小，比较节约硬盘空间
  缺点： 复制到其他电脑上或者软件包位置改变，就需要修改相对应的路径
- 一般选择第二个选项

第二个栏中配置的是生成文件时候的选项，四个开关分别为：

- 为每个启用的外设生成独立的.c/.h配置文件（如果不开则是在main.c中）
- 重新生成的时候备份会被覆盖的文件
- 在重新生成文件的时候保留用户代码
- 删除再次生成时不需要的文件

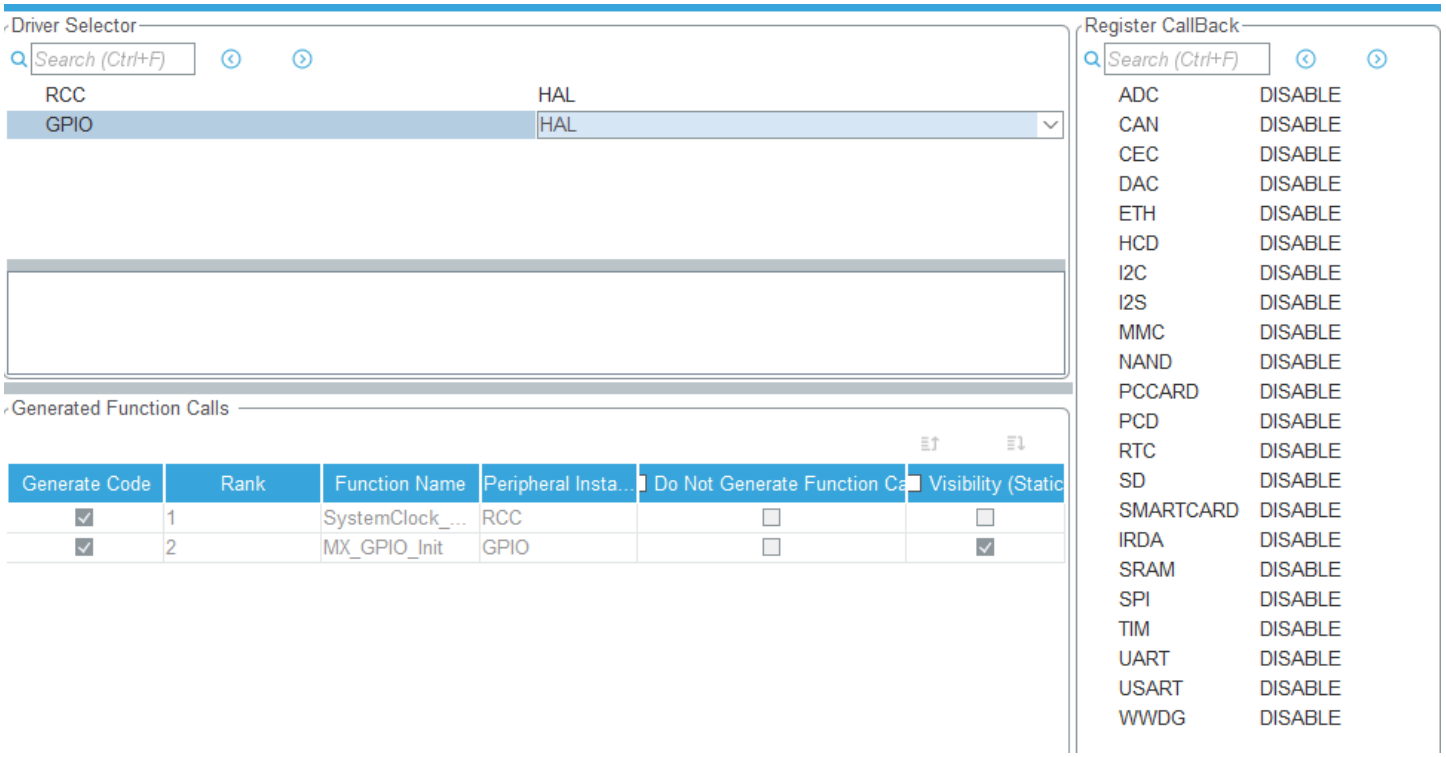
第三个栏是HAL库设置

- 释放所有引脚（降低功耗）
- 启用HAL库中所有的参数检查

第四个栏不要轻易使用，会使CubeMX卡死

第五个栏是模板设置，一般不用

- 高级设置




左上角让你选择使用HAL库还是LL库，一般是HAL库

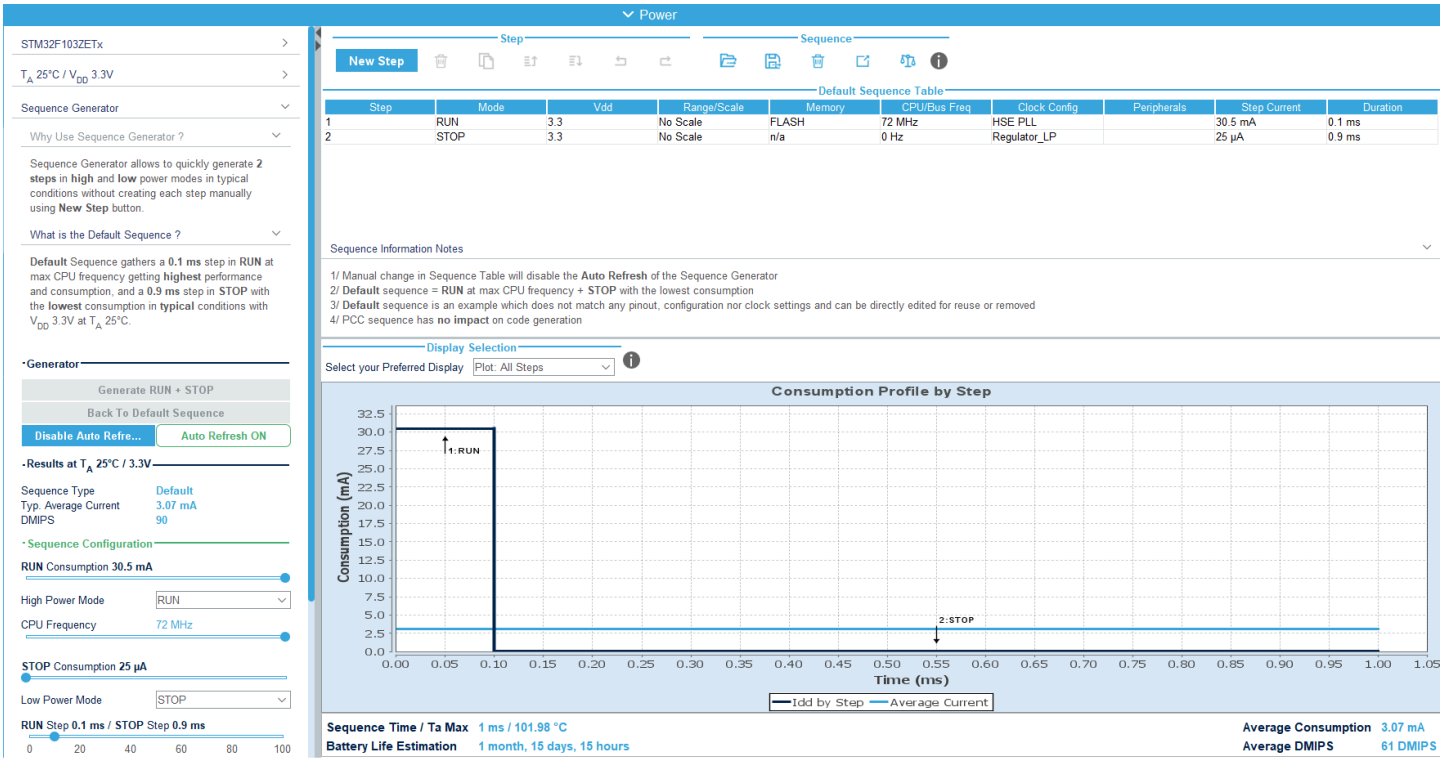
中间让你选择这些外设的初始化是否调用初始化函数，不调用则默认不会初始化这个外设。

右侧会让你选择回调函数是使用默认还是自己设置，学到中断之后可能会用到。

## 工具




工程对比，一般不用

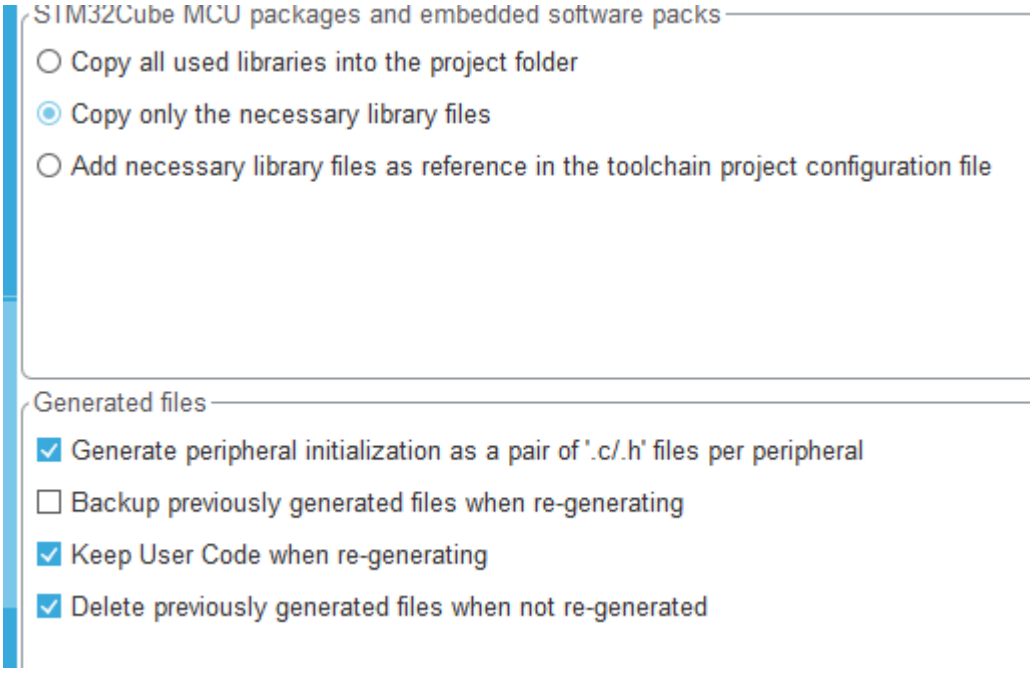



功耗计算，一般不用

## 生成代码结构剖析

Project中Project Settings的Application Structure中选择Advanced（默认）

Code Generator中这么配置




点击 GENERATE CODE，生成代码，打开文件夹，生成的代码结构如下



| | | | |
|---|---|---|---|
| cmake | 2026/8/27 5:34 | 文件夹 | |
| Core | 2026/8/27 5:34 | 文件夹 | |
| Drivers | 2026/8/27 5:34 | 文件夹 | |
| .mxproject | 2026/8/27 5:34 | MXPROJECT 文件 | 7 KB |
| CMakeLists.txt | 2026/8/27 5:34 | 文本文档 | 2 KB |
| CMakePresets.json | 2026/8/27 5:34 | JSON 源文件 | 1 KB |
| light-magic.ioc | 2026/8/27 5:34 | STM32CubeMX | 5 KB |
| startup_stm32f103xe.s | 2026/8/27 5:34 | Assembler Source | 12 KB |
| STM32F103XX_FLASH.ld | 2026/8/27 5:34 | LD 文件 | 9 KB |

Core: 程序入口，各种外设的初始化文件

Drivers：HAL库，Cortex-M驱动

cmake：存放编译工具链脚本 + Cube 自动生成的源码列表配置，不要手动修改里面的文件，重新生成工程时会直接覆盖

下一个章节中将会配合开发板演示如何使用CubeMX和CLion行联合开发

**-- By Scum**

**QQ: 3014827510**